## Pass 4:

I=4, temp=a[4]=3, j=3
A[4]=a[3] A[3]=a[2]
inner loop
A[2]=3 outer loop
1 , 2, 3 , 4, , 5 (SORTED LIST)

## Heap Sort

- Heap sort is a comparison based sorting technique based on Binary Heap data structure. It is similar to selection sort where we first find the maximum element and place the maximum element at the end. We repeat the same process for remaining element.
- A Binary Heap is a complete Binary Tree where items are stored in a special order such that value in a parent node is greater (or smaller) than the values in its two children nodes. If the parent node is greater than the children nodes it is called a max heap and if the parent node is smaller than the children nodes it is called a min heap. The heap can be represented by binary tree or array.
- We use an array because Binary Heap is a Complete Binary Tree, it can be easily represented as array and array based representation is space efficient. If the parent node is stored at index I, the left child can be calculated by 2 * I + 1 and right child by 2 * I + 2 (assuming the indexing starts at 0).

## Procedure:

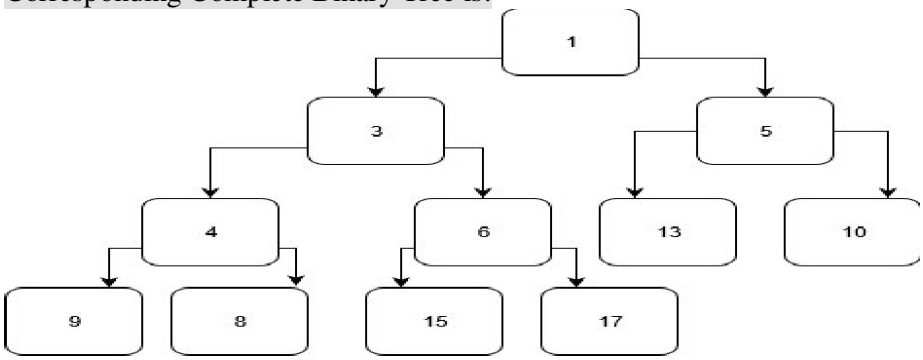
- Initially build a max heap of elements in Array.(Refer 2nd point for what is a Max Heap)
- The root element, that is Array [1], will contain maximum element of Array (since it is a max heap).
- After that, swap this element with the last element of Array and heapify the max heap excluding the last element which is already in its correct position and then decrease the length of heap by one.
- Repeat the step 2, until all the elements are in their correct position.

## Building a Max heap:

Consider an array,

Array = {1, 3, 5, 4, 6, 13, 10, 9, 8, 15, 17}

Corresponding Complete Binary Tree is:

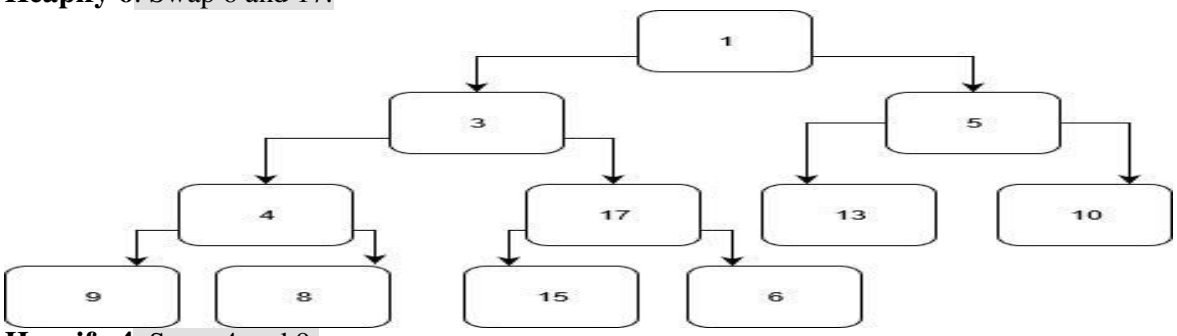



***The task to build a Max-Heap from above array***

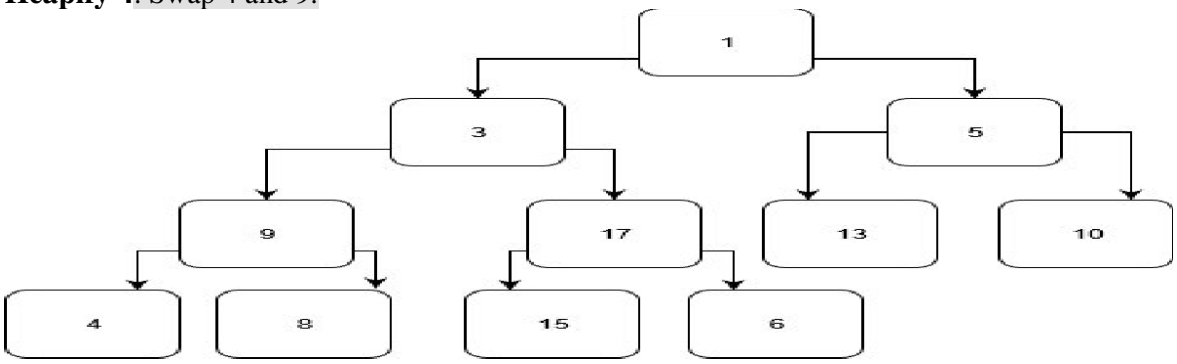
Total Nodes = 11.

To build the heap, heapify only the nodes: [1, 3, 5, 4, 6] in **reverse order** as the rest of the nodes are leaf nodes [13, 10, 9, 8, 15, 17].

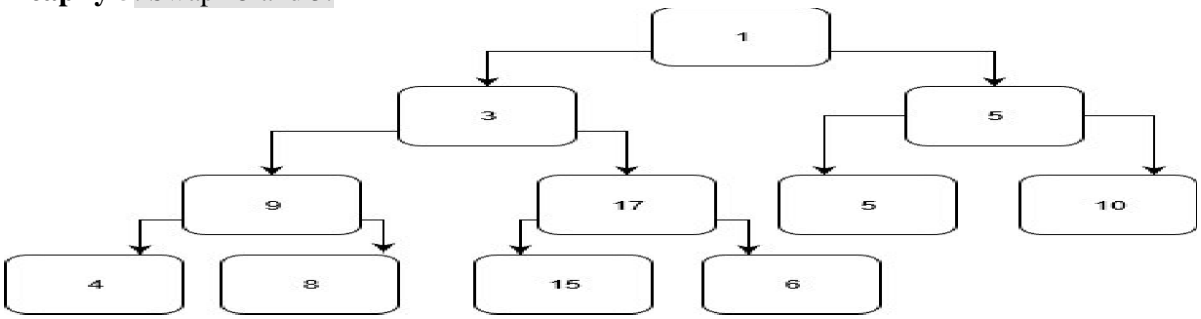
**Heapify 6**: Swap 6 and 17.



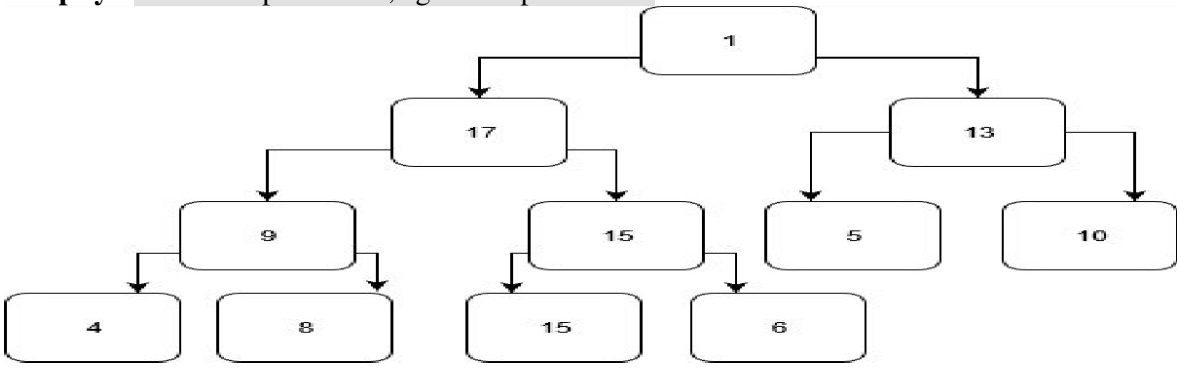

**Heapify 4**: Swap 4 and 9.



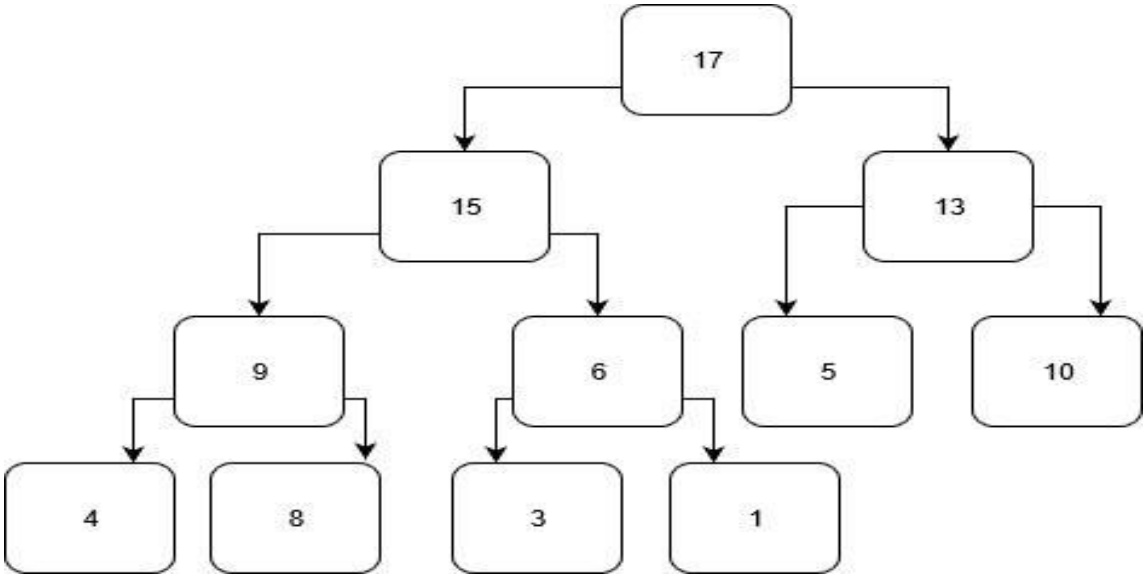

**Heapify 5**: Swap 13 and 5.




**Heapify 3**: First Swap 3 and 17, again swap 3 and 15.



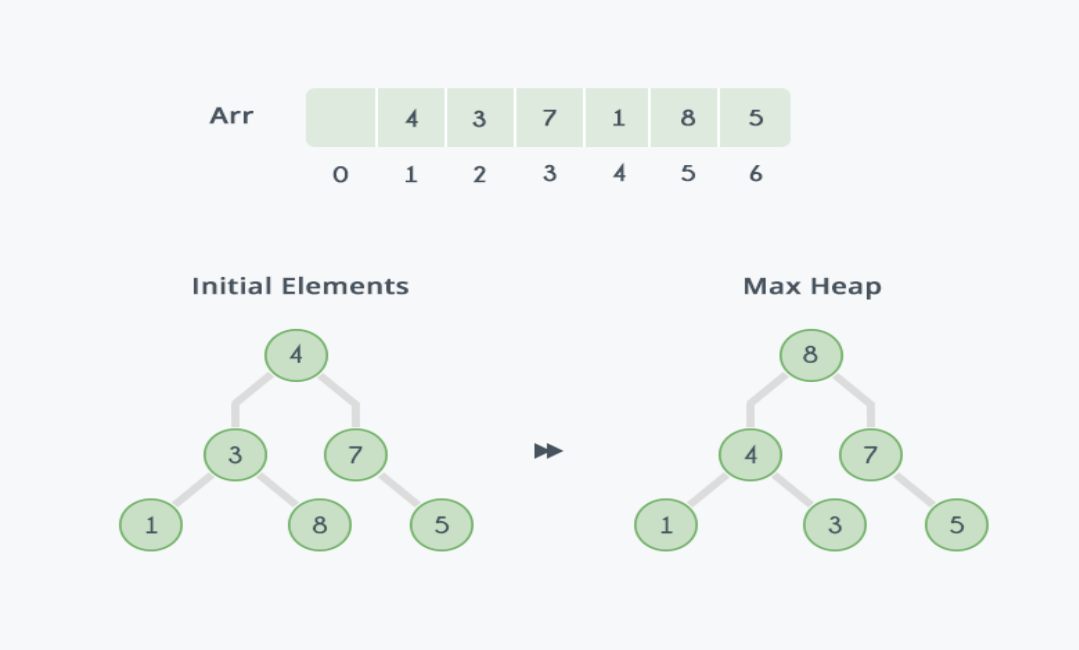
**Heapify 1**: First Swap 1 and 17, again swap 1 and 15,
Finally swap 1 and 6.



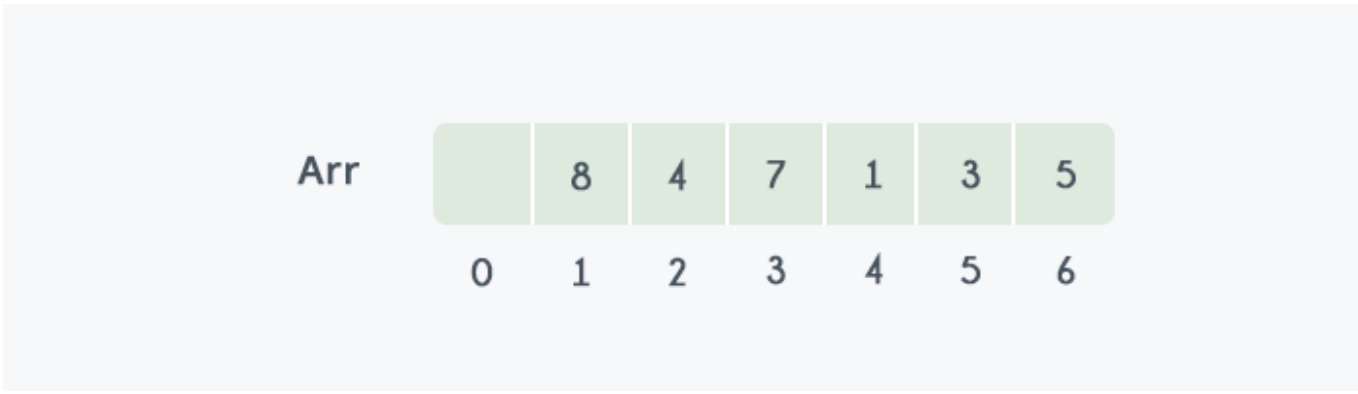

Hence the elements are arranged in a max heap.

**Example for heap sort:**

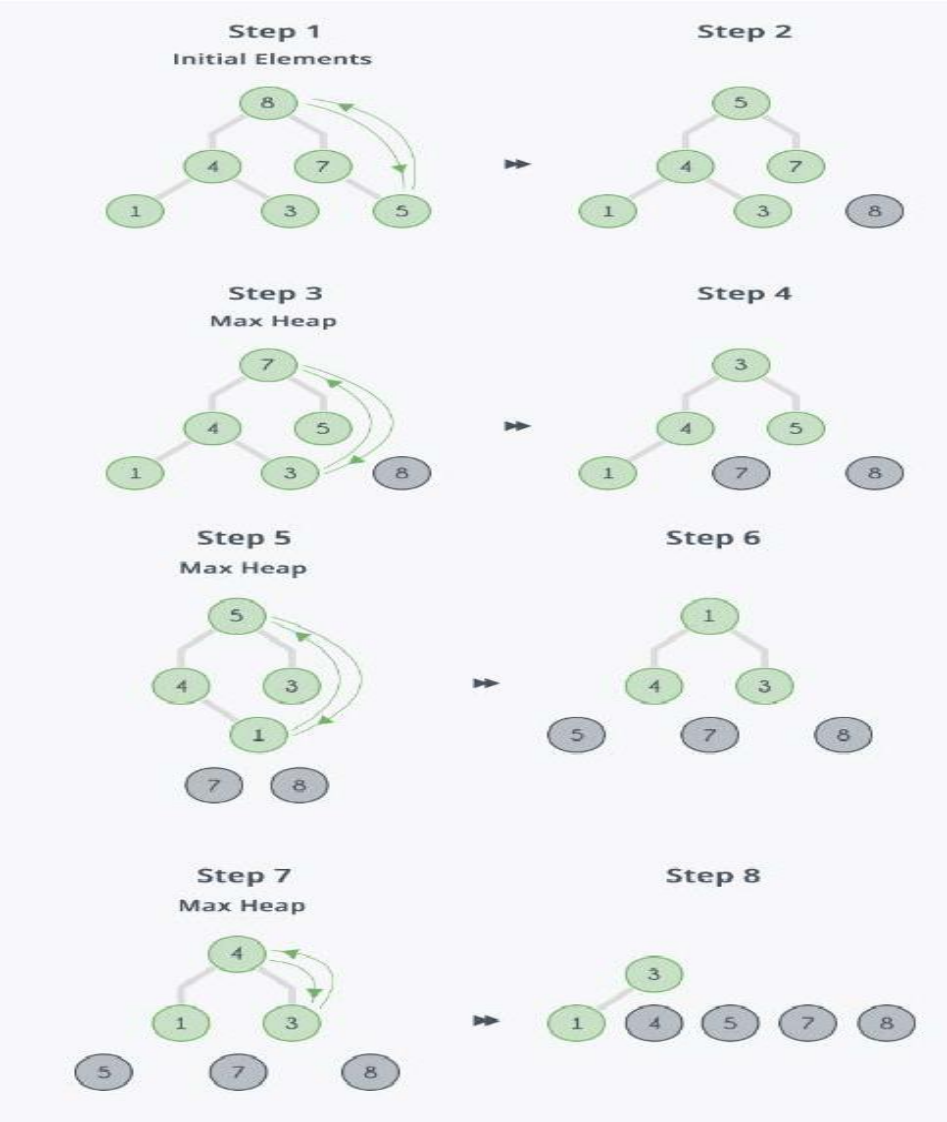
In the diagram below, initially there is an unsorted array having 6 elements and then max-heap will be built.



After building max-heap, the elements in the array will be:




Step 1: 8 is swapped with 5.
Step 2: 8 is disconnected from heap as 8 is in correct position now.
Step 3: Max-heap is created and 7 is swapped with 3.
Step 4: 7 is disconnected from heap.
Step 5: Max heap is created and 5 is swapped with 1.
Step 6: 5 is disconnected from heap.
Step 7: Max heap is created and 4 is swapped with 3.
Step 8: 4 is disconnected from heap.
Step 9: Max heap is created and 3 is swapped with 1.
Step 10: 3 is disconnected.

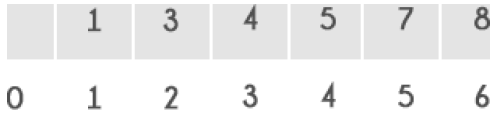






After all the steps, we will get a sorted array.

Arr



# External Sorting

External sorting is a term for a class of sorting algorithms that can handle massive amounts of data. External sorting is required when the data being sorted do not fit into the main memory of a computing device (usually RAM) and instead they must reside in the slower external memory (usually a hard drive). External sorting typically uses a hybrid sort-merge strategy. In the sorting phase, chunks of data small enough to fit in main memory are read, sorted, and written out to a temporary file. In the merge phase, the sorted sub-files are combined into a single larger file.

One example of external sorting is the external merge sort algorithm, which sorts chunks that each fit in RAM, then merges the sorted chunks together. We first divide the file into **runs** such that the size of a run is small enough to fit into main memory. Then sort each run in main memory using merge sort sorting algorithm. Finally merge the resulting runs together into successively bigger runs, until the file is sorted.

# Merge Sort

Merge sort is a divide-and-conquer algorithm based on the idea of breaking down a list into several sub-lists until each sub list consists of a single element and merging those sub lists in a manner that results into a sorted list.

## Idea:

- Divide the unsorted list into N sub lists, each containing 1 element.
- Take adjacent pairs of two singleton lists and merge them to form a list of 2 elements. N will now convert into N/2 lists of size 2.
- Repeat the process till a single sorted list of obtained.

## Time Complexity:

The list of size N is divided into a max of logN parts, and the merging of all sub lists into a single list takes O (N) time, the worst case run time of this algorithm is O (NLogN).

## How Merge Sort Works?

To understand merge sort, we take an unsorted array as the following −



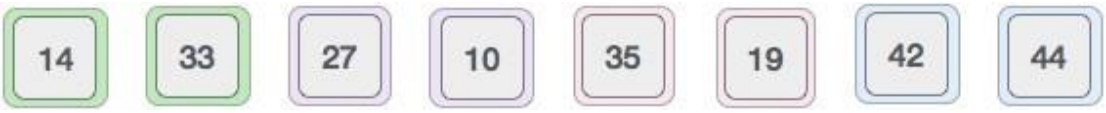

We know that merge sort first divides the whole array iteratively into equal halves unless the atomic values are achieved. We see here that an array of 8 items is divided into two arrays of size 4.




This does not change the sequence of appearance of items in the original. Now we divide these two arrays into halves.

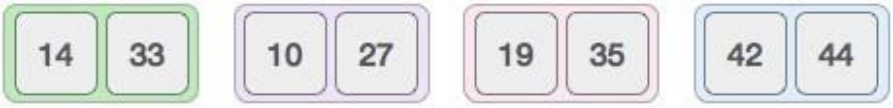



We further divide these arrays and we achieve atomic value which can no more be divided.




Now, we combine them in exactly the same manner as they were broken down. Please note the color codes given to these lists.

We first compare the element for each list and then combine them into another list in a sorted manner. We see that 14 and 33 are in sorted positions. We compare 27 and 10 and in the target list of 2 values we put 10 first, followed by 27. We change the order of 19 and 35 whereas 42 and 44 are placed sequentially.



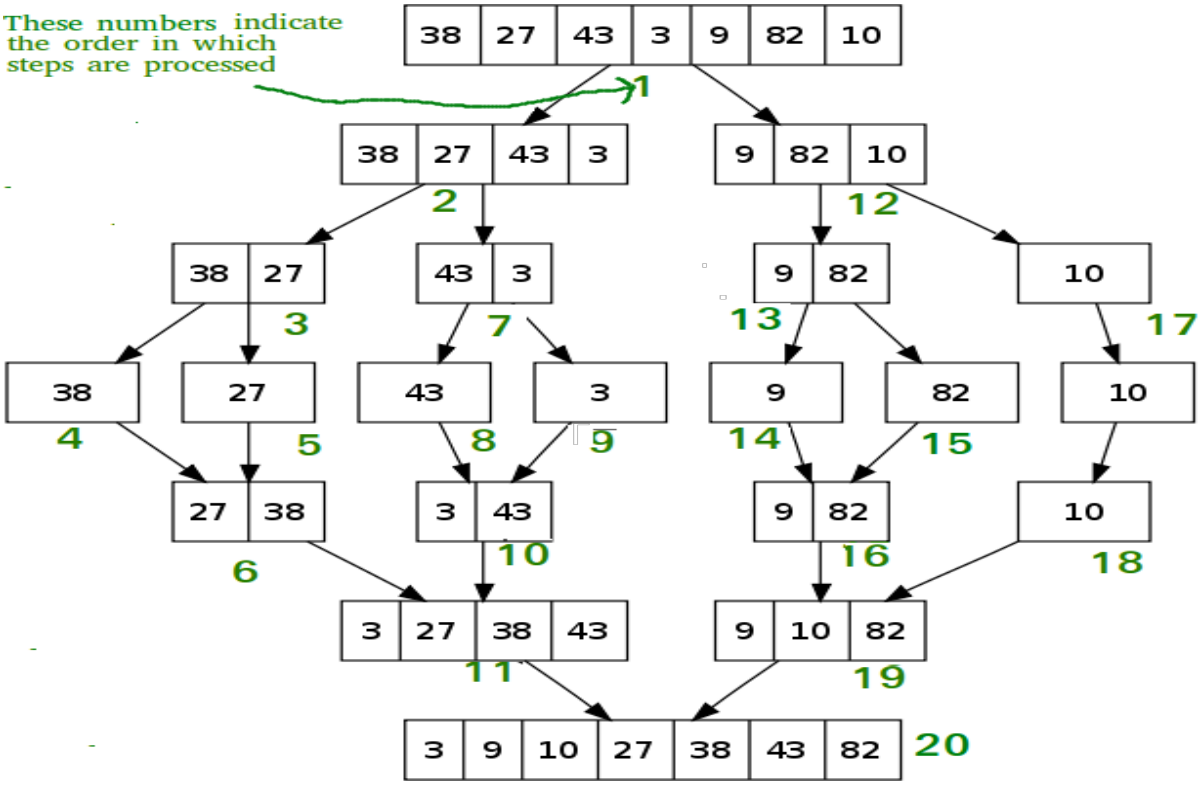

In the next iteration of the combining phase, we compare lists of two data values, and merge them into a list of found data values placing all in a sorted order.




After the final merging, the list should look like this −



Consider another example,




## Applications of Merge Sort

- Merge sort is useful for sorting linked lists in O (nlogn) time.
- Inversion Count Problem.
- Used in External Sorting.